

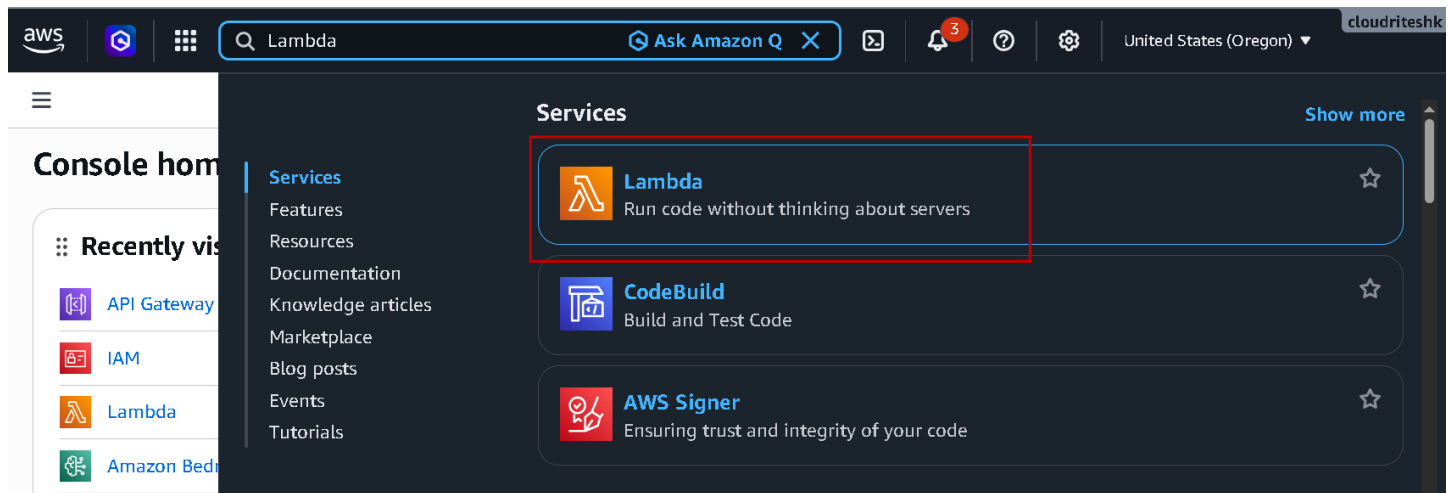
# Code Generator

## Summary: -

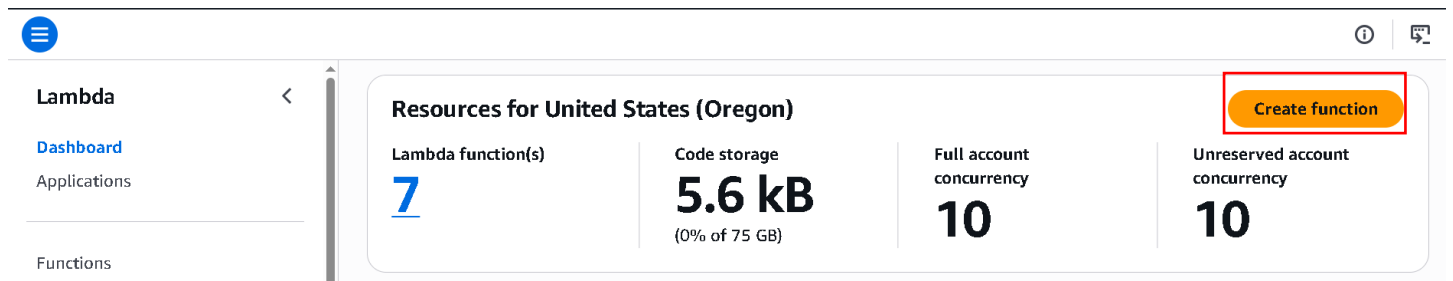
You created a smart code generation system using Amazon Bedrock that can generate programming code from user instructions. Users can send requests like creating a Python program, and the system automatically generates the code and stores it in Amazon S3. In short, you built an AI tool that can create and save code automatically from simple user requests.

## Now Let's Create our project: -

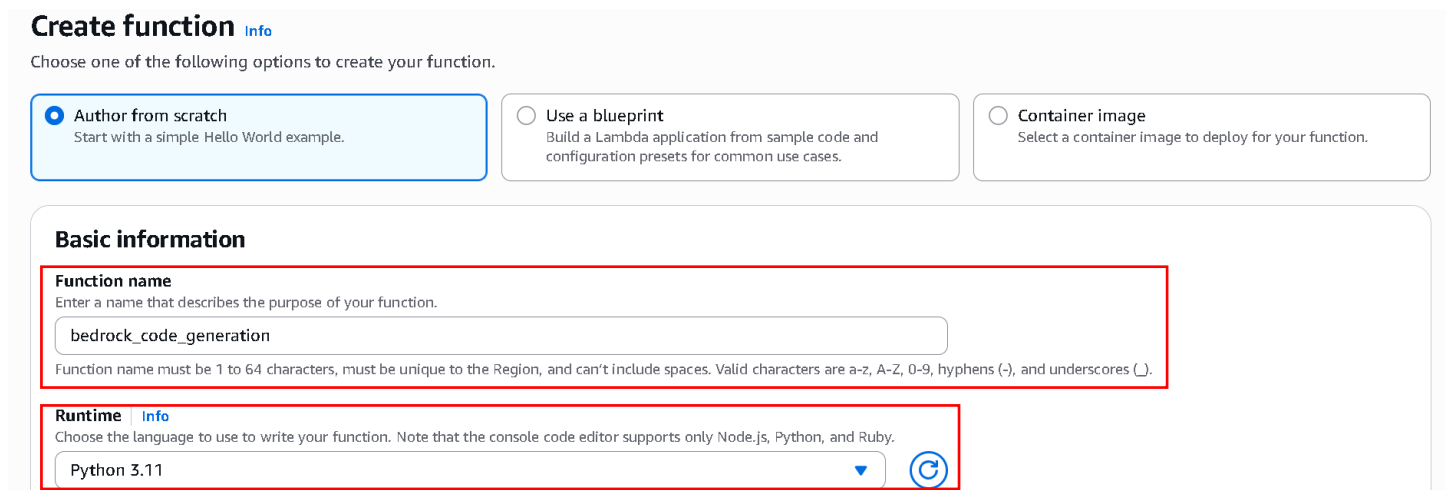
- Open your AWS Console With IAM User Account.



- Search and open Lambda.



- Now just click Create a function.



- Enter function name bedrock\_code\_generation.
- And select Runtime python 3.11.

### ► Additional settings

Customize your function architecture, execution role, HTTP(S) endpoints, tenant isolation mode, VPC, code signing, KMS key, and tags.

✕ Cannot add or remove after creation

Cancel

Create function

- Now simply scroll down and click Create function.

## bedrock\_code\_generation

### ▼ Function overview [Info](#)

Diagram

Template



bedrock\_code\_generation



Layers

(0)

+ Add trigger

+ Add destination

Code

Test

Monitor

Configuration

Aliases

Versions

- Our lambda function is created.
- Now click Configuration.
- And scroll down.

Code

Test

Monitor

Configuration

Aliases

Versions

### Configuration

General configuration

Triggers

Permissions

Destinations

Function URL

### Execution role



[View role document](#)

[Edit](#)

#### Role name

[bedrock\\_code\\_generation-role-jy2t52b5](#)

### Resource summary

To view the resources and actions that your function has permission to access, choose a service.



Amazon CloudWatch Logs

3 actions 2 resources



- In configuration select Permissions.
- And then click Role name.

## bedrock\_code\_generation-role-jy2t52b5 [Info](#)

[Delete](#)

### Summary

[Edit](#)

#### Creation date

May 11, 2026, 13:23 (UTC+05:30)

#### Last activity

-

#### ARN

arn:aws:iam::463646775279:role/service-role/bedrock\_code\_generation-role-jy2t52b5

#### Maximum session duration

1 hour

### Permissions

[Trust relationships](#)[Tags](#)[Last Accessed](#)[Revoke sessions](#)[Attach policies](#)[Create inline policy](#)[Add permissions](#)

### Permissions policies (1) [Info](#)

[Simulate](#)[Remove](#)

- Now click Add permissions and then select Attach policies.

### Other permissions policies (1/2181)



Filter by Type

All types

5 matches

&lt; 1 &gt;



| <input type="checkbox"/>            | Policy name                                             | Type                       | Description                                |
|-------------------------------------|---------------------------------------------------------|----------------------------|--------------------------------------------|
| <input checked="" type="checkbox"/> | <a href="#">AdministratorAccess</a>                     | AWS managed - job function | Provides full access to AWS services a...  |
| <input type="checkbox"/>            | <a href="#">AdministratorAccess-Amplify</a>             | AWS managed                | Grants account administrative permis...    |
| <input type="checkbox"/>            | <a href="#">AdministratorAccess-AWSElasticBeanstalk</a> | AWS managed                | Grants account administrative permis...    |
| <input type="checkbox"/>            | <a href="#">AWSAuditManagerAdministratorAccess</a>      | AWS managed                | Provides administrative access to ena...   |
| <input type="checkbox"/>            | <a href="#">AWSManagementConsoleAdministratorAcc...</a> | AWS managed - job function | Provides full access to configure and c... |

[Cancel](#)[Add permissions](#)

- Now Search Administrator Access and select it.
- If this is a Real work Environment we decide what function our function need and we give only that Access.
- And simply click Add permission.
- And close the tab and return back to lambda function.

### Code

[Test](#)[Monitor](#)[Configuration](#)[Aliases](#)[Versions](#)

### Code source [Info](#)

[Open in Visual Studio Code](#)

← → bedrock\_code\_generation

EXPLORER

BEDROCK\_CODE\_GENERATION

lambda\_function.py

DEPLOY Undeployed

Deploy (Ctrl+Shift+U)

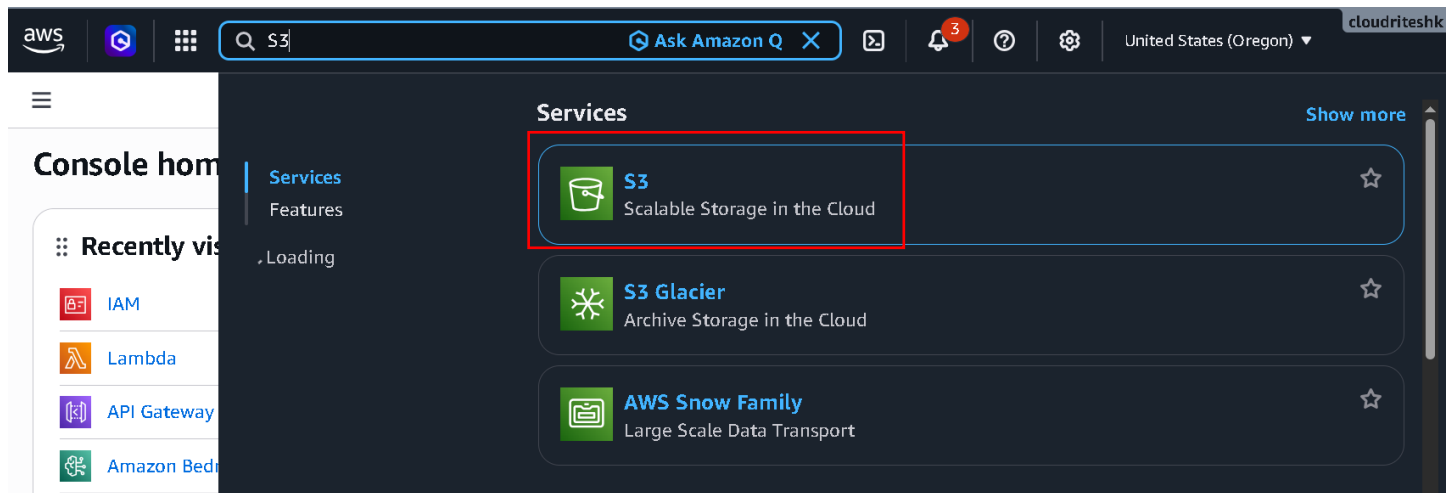
Test (Ctrl+Shift+I)

lambda\_function.py

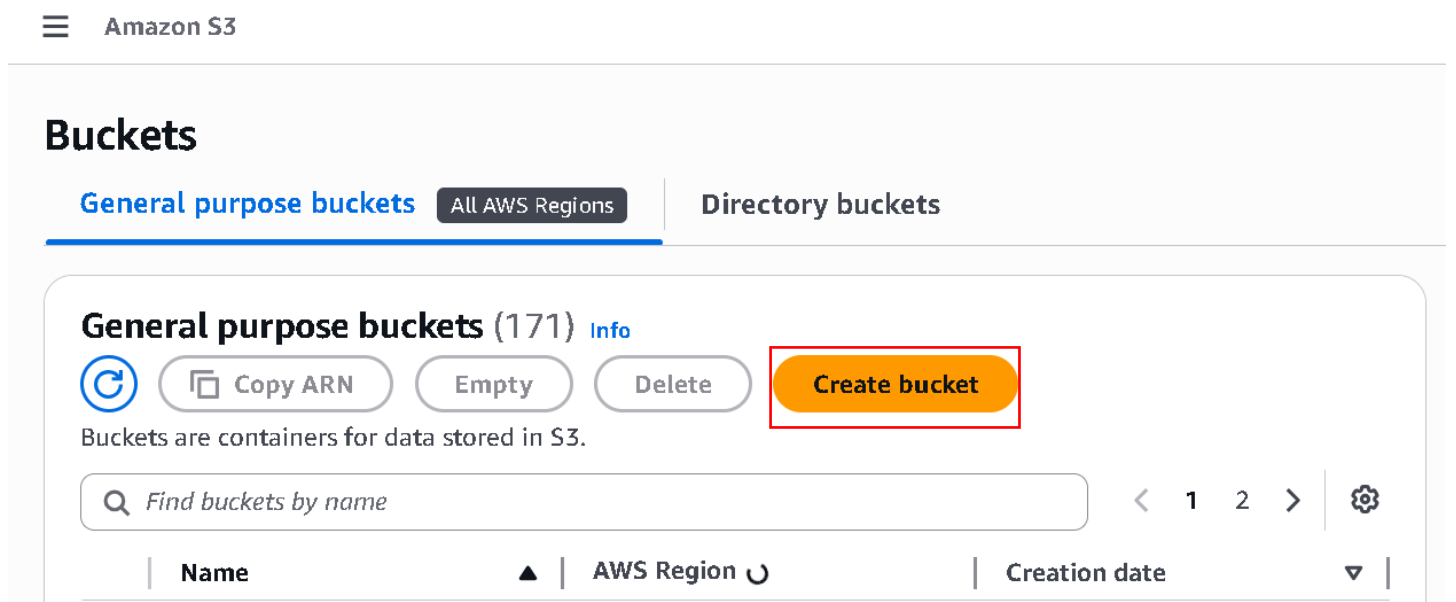
```
45 def lambda_handler(event, context):
46
47     if generated_code:
48         current_time = datetime.now().strftime('%H%M%S')
49         s3_key = f'code-output/{current_time}.py'
50         s3_bucket = 'bedrock-course-bucket'
51
52         save_code_to_s3_bucket(generated_code, s3_bucket, s3_key)
53
54     else:
55         print("No code was generated")
56
57     return {
```

- Replace the previous written code with the code that provided in Resource.
- Now we need S3 Bucket.

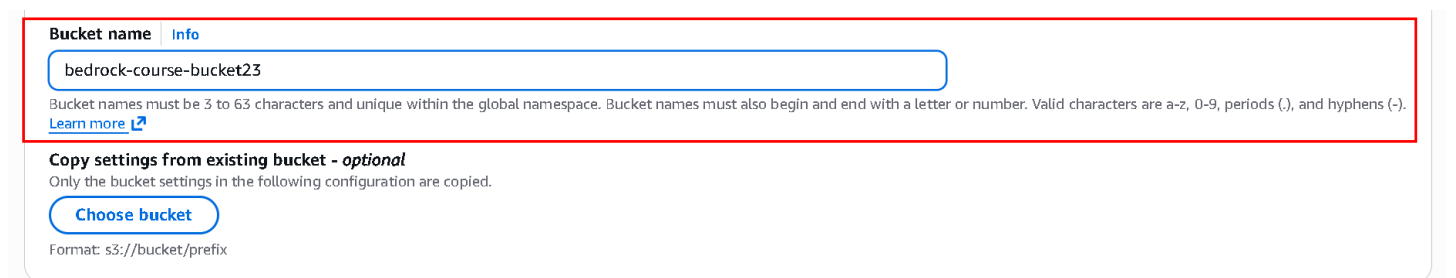
- So there we can store our output.
- Open Aws console in new tab.



- Now search S3 and open it.



- Now click Create Bucket.



- Now scroll down and Enter Bucket name.
- And leave Else default and scroll down.

## ► Advanced settings

After creating the bucket, you can upload files and folders to the bucket, and configure additional bucket settings.

Cancel

Create bucket

- Now click Create bucket.

Amazon S3 > Buckets > bedrock-course-bucket23

bedrock-course-bucket23 [Info](#)

Objects

Metadata

Properties

Permissions

Metrics

Management

File systems - new

Access Point

Objects (0)



Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

- Now copy this bucket name.
- And go back to your lambda function tab.

Code

Test

Monitor

Configuration

Aliases

Versions

Code source [Info](#)

Open in Visual Studio Code

Update



bedrock\_code\_generation



EXPLORER



lambda\_function.py



Deploy (Ctrl+Shift+U)



Test (Ctrl+Shift+I)



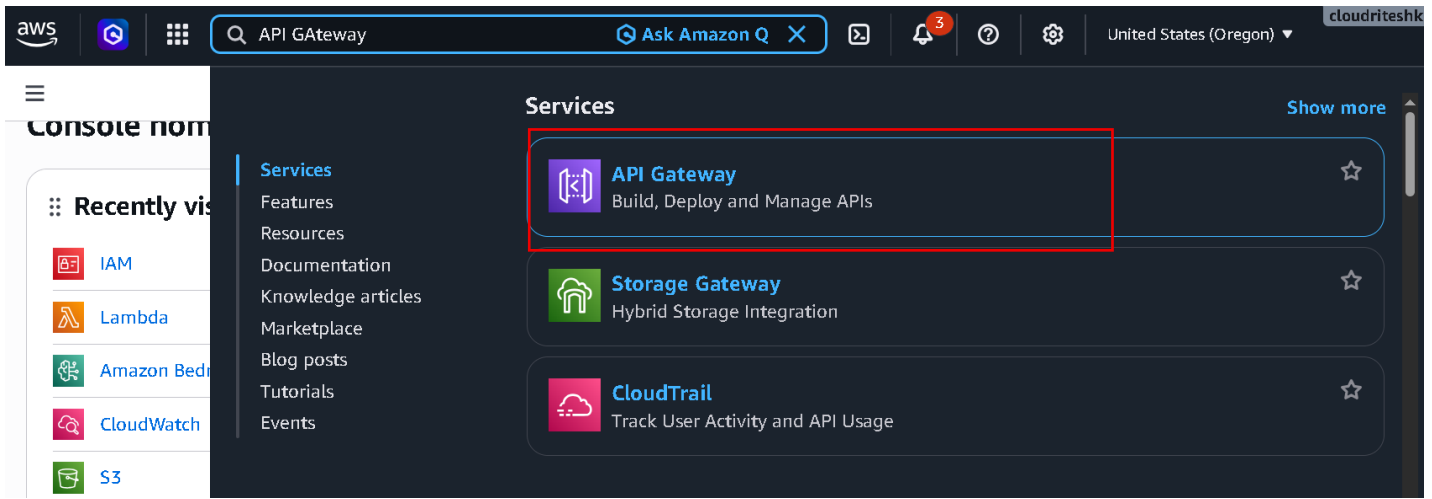
TEST EVENTS [NONE SELECTED]

+ Create new test event

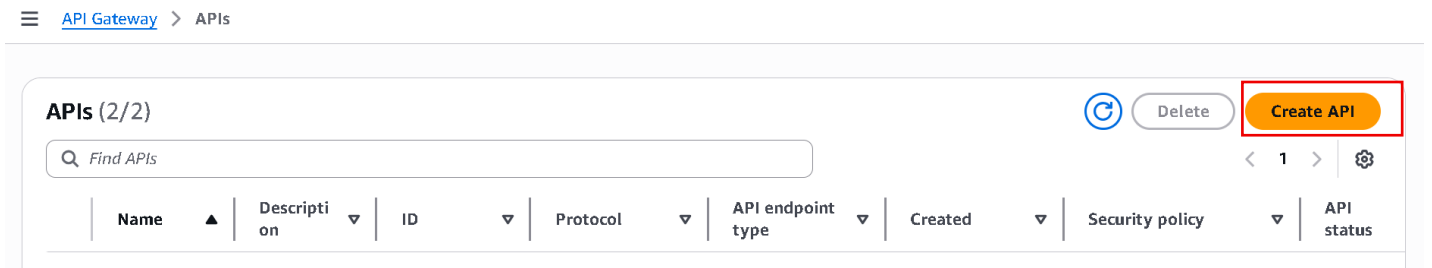
lambda\_function.py

```
45 def lambda_handler(event, context):
52
53     if generated_code:
54         current_time = datetime.now().strftime('%H%M%S')
55         s3_key = f'code-output/{current_time}.py'
56         s3_bucket = 'bedrock-course-bucket23'
57
```

- Now in your code paste this bucket name on line no 56.
- And simply click on deploy.
- Open Aws console in new tab and search API gateway.

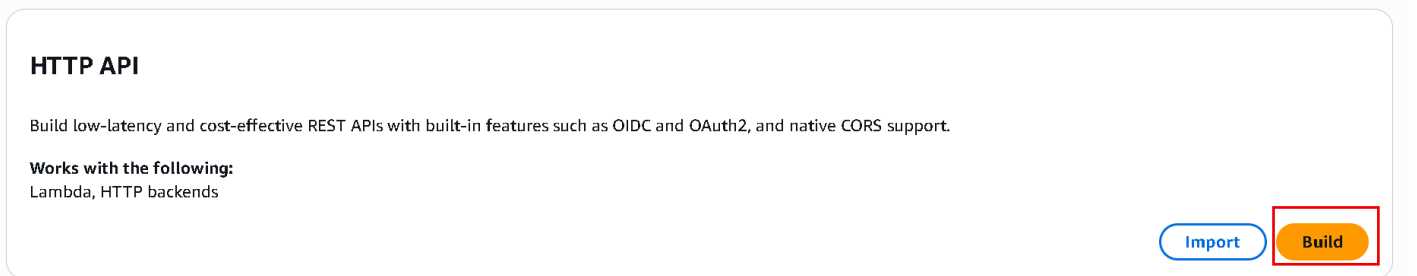


- Now search and open it API Gateway.



- Now click on create API.

## Choose an API type [Info](#)



- Now click Build.

## Configure API

### API details

#### API name

An HTTP API must have a name. The name is a non-unique value you use to identify and organize your APIs. To programmatically refer to this API, use the API ID that API Gateway generates for you.

bedrock\_api

#### IP address type [Info](#)

Select the type of IP addresses that can invoke the default endpoint for your API. You don't need to redeploy your API for the update to take effect.

- ☒ **IPv4**  
Includes only IPv4 addresses.
- ☐ **Dualstack**  
Includes IPv4 and IPv6 addresses.

- Enter Api name bedrock\_api.
- Now simply scroll down and leave else default Everything.

## Integrations (0) [Info](#)

Specify the backend services that your API will communicate with. These are called integrations. For a Lambda integration, API Gateway invokes the Lambda function and responds with the response from the function. For an HTTP integration, API Gateway sends the request to the URL that you specify and returns the response from the URL.

[Add integration](#)

[Cancel](#)

[Review and create](#)

[Next](#)

- Now simply click Review and Create.

## Stages

[Edit](#)

### Stages

- \$default (Auto-deploy: enabled)

[Cancel](#)

[Previous](#)

[Create](#)

- Now simply click Create.

✓ Successfully created API bedrock\_api (az67be7t97).

✕

## Routes

Stage: - ▼

[Deploy](#)

### Routes for bedrock\_api

[Create](#)

🔍 Search

Choose a route.

- Now simply click Create.

GET PUT PATCH HEAD OPTIONS DELETE ANY POST

POST API bedrock\_api (az67be7t97).

Method Info

Enter a path to create a route. You can also specify one \$default route per API. The \$default route is invoked when the request to the API

/code-generation

Cancel Create

- Select POST and Enter /code-generation
- And simply click Create.

## Routes

Routes for bedrock\_api Create

Search

▼ /code-generation

POST

- Now simply click POST.

Routes Stage: - ▼

Routes for bedrock\_api Create

Search

▼ /code-generation

POST

Route details Delete

POST /code-generation (ID: hvuonlk)

ARN  
arn:aws:apigateway:us-west-2::apis/az67be7t97/routes/hvuonlk

Authorization  
Authorizers protect your API against unauthorized requests. Routes with no authorization attached are open.  
No authorizer attached to this route. Attach authorization

Integration  
The integration is the backend resource that this route calls when it receives a request.  
No integration attached to this route. Attach integration



- Now click Attach Integration.

## Integrations

Stage: - ▼

Attach integrations to routes

Manage integrations

### Routes for bedrock\_api

Q Search

▼ /code-generation

POST

### Integration details

POST /code-generation (ID: hvuonlk)

You have no integrations configured for this API. Create an integration to attach it.

Create and attach an integration

- Now simply click Create and attach an integration.

## Create an integration

### Attach this integration to a route

Q POST /get-agents-response

×

### Integration target

Integration type

Lambda function ▼

- Integration target Lambda function.

### Integration details

#### Integration target

Choose the Lambda function that you want to use.

AWS Region

us-west-2 ▼

► Advanced settings

Description - optional

arn:aws:lambda:us-west-2:463646775279:function:bedrock\_code\_generation

arn:aws:lambda:us-west-2:463646775279:function:bedrock\_code\_generation

Q 1:aws:lambda:us-west-2:463646775279:function:bedrock\_code\_generation

×

- Now select your Lambda function that you created.

### Invoke permissions

API Gateway requests permission to invoke your Lambda function by granting this API permission in your Lambda function's resource policy. If you don't want to modify the resource policy, you can provide an invoke role instead. API Gateway uses this role to invoke the Lambda function.

☒ Grant API Gateway permission to invoke your Lambda function

Cancel

Create

- Now scroll down.
- And click Create.

**API Gateway**

APIs  
Custom domain names  
Domain name access associations  
VPC links

API: bedrock\_api...(az67be7t97)

▼ **Develop**  
Routes  
Authorization  
**Integrations**

**Integrations**

Attach integrations to routes | Manage integrations

**Routes for bedrock\_api**

Search

▼ /code-generation

POST **AWS Lambda**

**Integration details for route**

POST /code-generation (hvuonlk)

**Lambda function**  
bedrock\_code\_generation (us-west-2)

**Integration ID**  
jrvxqtr

**Description**  
-

**Payload format version**  
The parsing algorithm for the payload sent to and returned from your Lambda function. [Learn more](#)

**Detach Integration** **Manage integration**

**Deploy**

- Now Click in left side created API.

**API Gateway** > **APIs** > bedrock\_api (az67be7t97)

▼ **Develop**  
Routes  
Authorization  
Integrations  
CORS  
Reimport  
Export

▼ **Deploy**  
**Stages**

▼ **Monitor**  
Metrics  
Logging

**Stages for bedrock\_api (1)**

Find resources

| Stage name                | Invoke URL                                                                                                                  | Attached deployment | Auto deploy |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------|---------------------|-------------|
| <a href="#">\$default</a> | <a href="https://az67be7t97.execute-api.us-west-2.amazonaws.com">https://az67be7t97.execute-api.us-west-2.amazonaws.com</a> | z1m0tj              | enabled     |

**Tags (0)**

Find resources

Key Value

**Manage tags**

**Deploy**

- Now this is our Invoke URL
- Now in nav bar inside Deploy click Stages.

**Stages**

Select a stage ▼ **Deploy**

**Stages for bedrock\_api**

**Create**

Search

☐ \$default

Select a stage

- Now click Create.

## Create Stage [Info](#)

### Stage details

Name

dev

Description

Enter description

- Now enter Stage name.
- simply scroll down and leave Everything as default.

### Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

No tags associated with the resource.

Add tag

Cancel

Create

- Now simply click Create.
- Now stage is Created go back to routes.

## Routes

Stage: - ▼

Deploy

### Routes for bedrock\_api

Create

Search

▼ /code-generation

### Route details

Delete

Edit

POST /code-generation (ID: hvuonlk)

ARN

arn:aws:apigateway:us-west-2::apis/az67be7t97/routes/hvuonlk

Authorization

Authorizers protect your API against unauthorized requests. Routes with no authorization attached are open.

No authorizer attached to this route.

Attach authorization

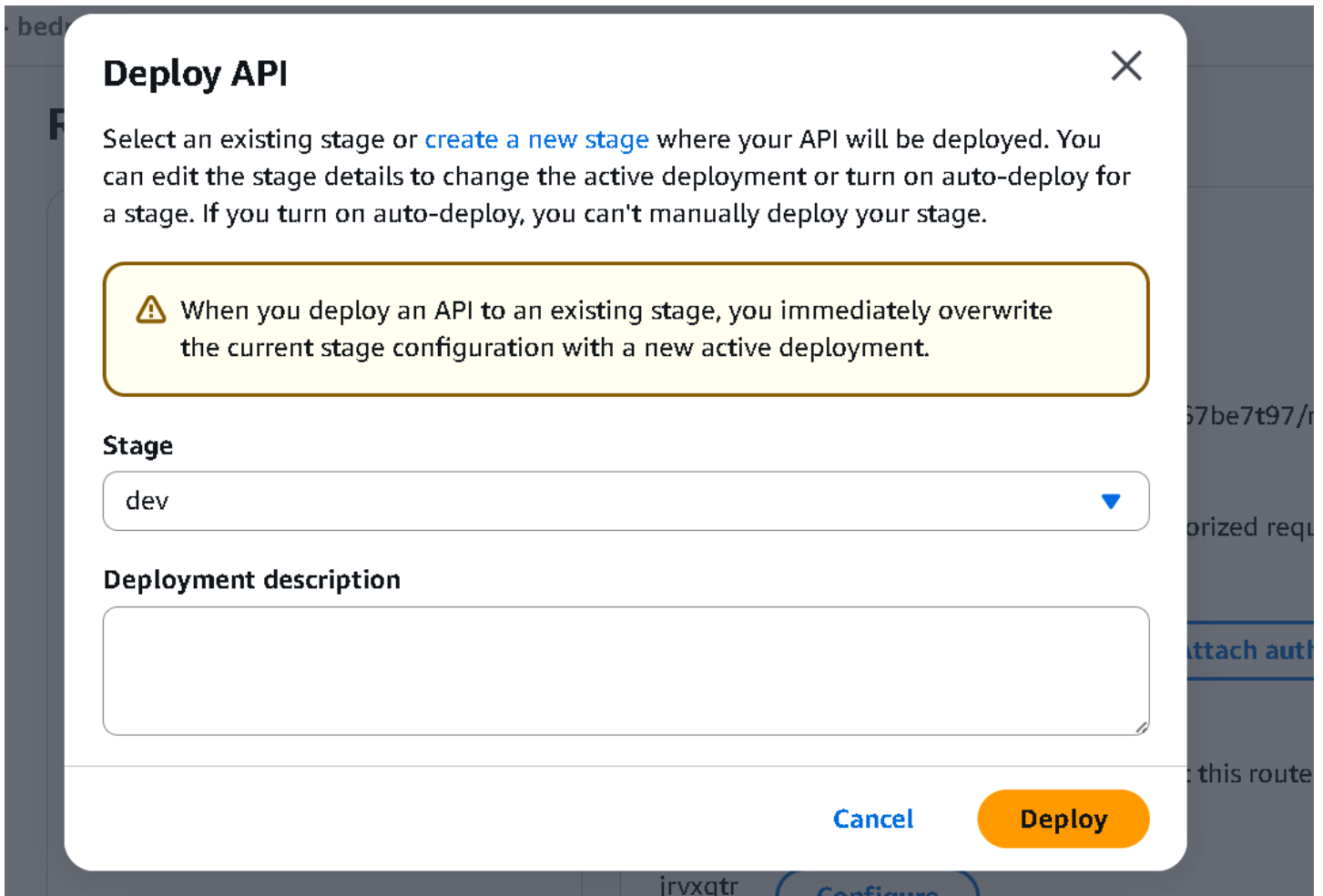
Integration

The integration is the backend resource that this route calls when it receives a request.

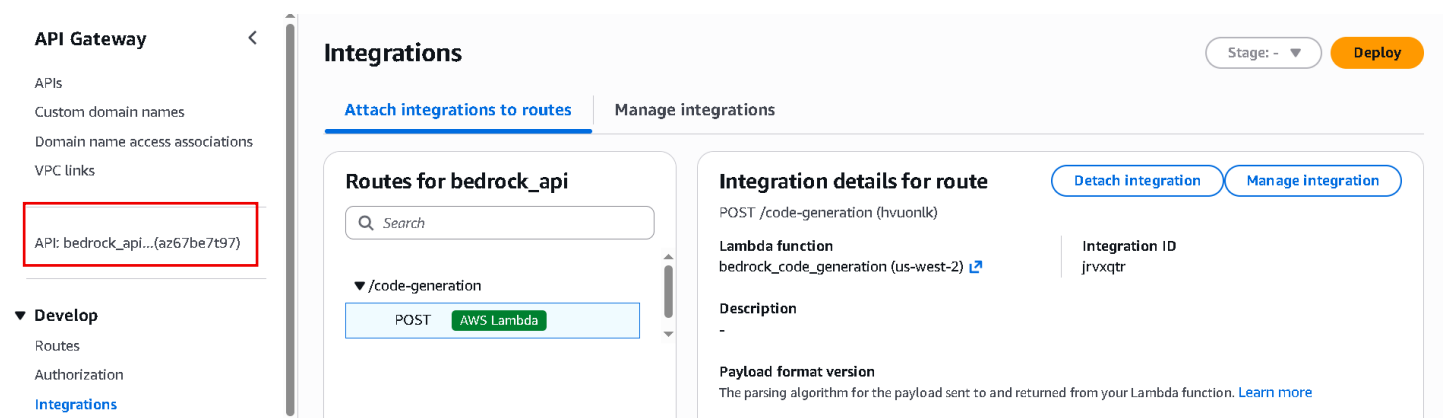
jrvxqtr

Configure

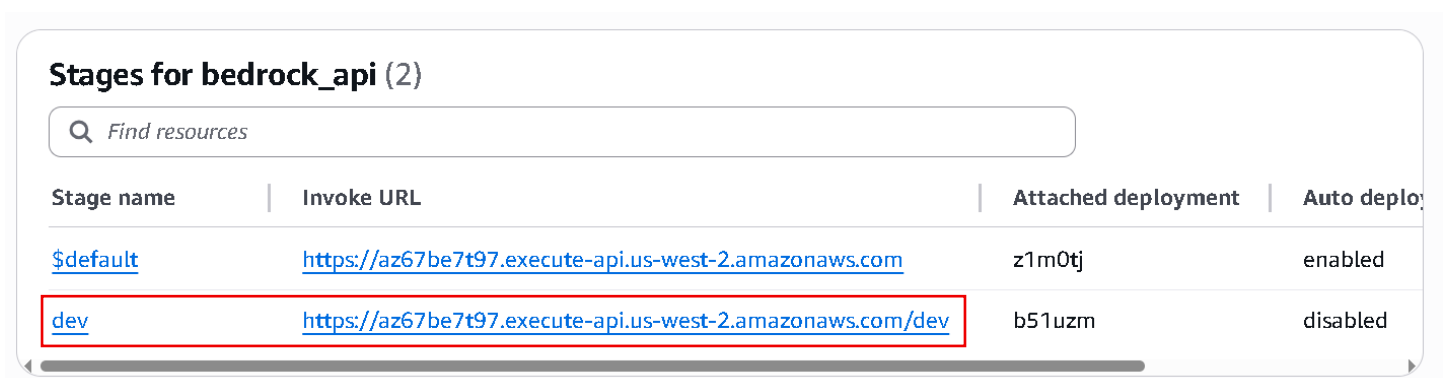
- And click Deploy.



- Select your created Stage and simply click Deploy.
- And click Create.

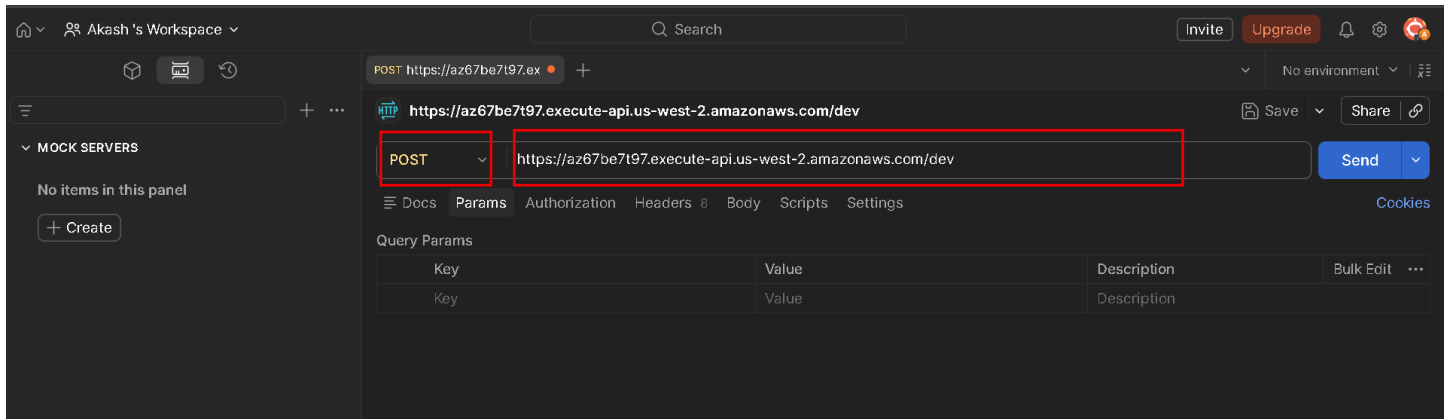


- Now again go inside this created Api.



- Now we have two invoke URL one is default and another is that we created now.
- Copy the dev invoke URL.

- And open postman in new tab.



- Now select POST and paste dev invoke URL.
- And again, go to API Gateway and go to Routes.

## Routes

Routes for bedrock\_api

Create

Search

▼ /code-generation

POST

Route details

POST /code-generation (ID: hvuonlk)

ARN

arn:aws:apigateway:us-west-2::apis/az67be7t97/routes/hvuonlk

Authorization

Authorizers protect your API against unauthorized requests. Routes with no authorization attached are open.

No authorizer attached to this route. [Attach authorization](#)

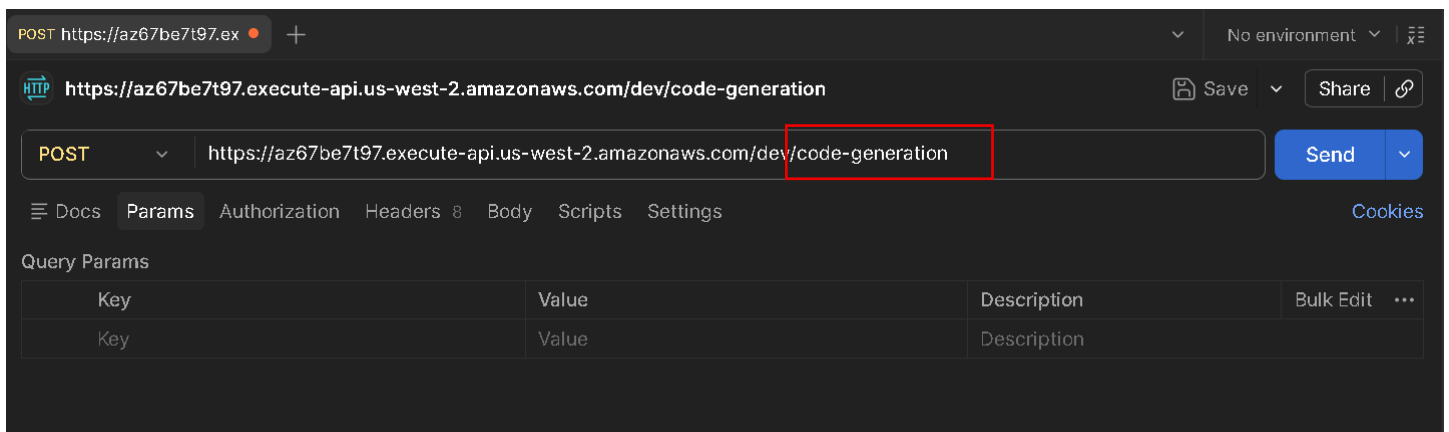
Integration

The integration is the backend resource that this route calls when it receives a request.

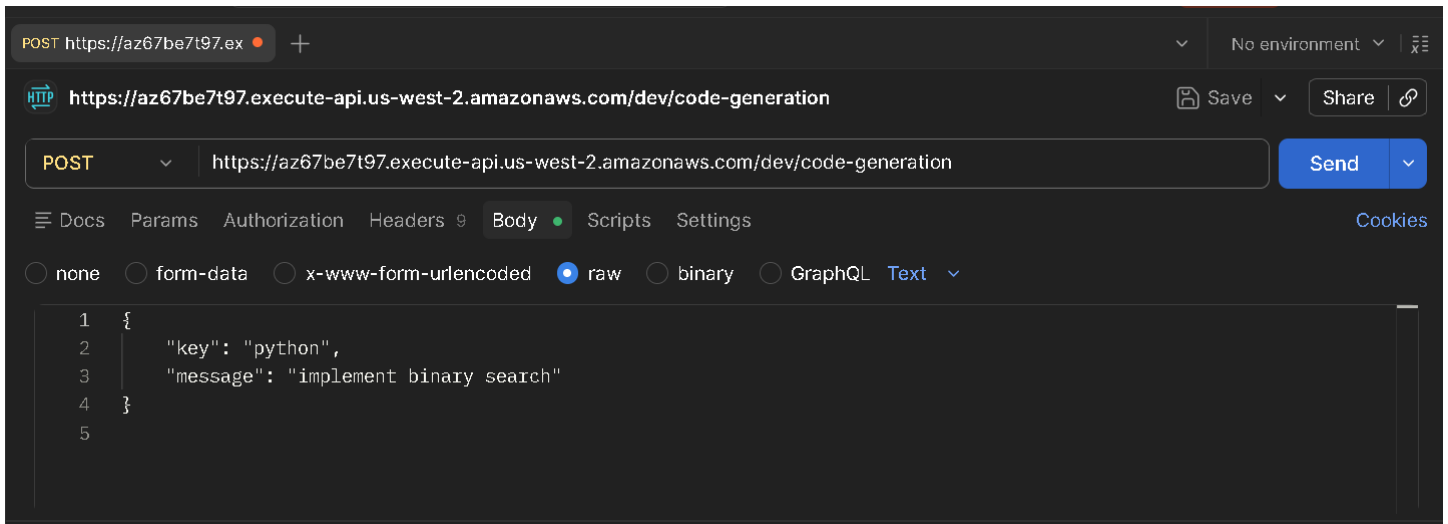
jrvxqtr [Configure](#)

Delete Edit

- Now copy the /code-generation.

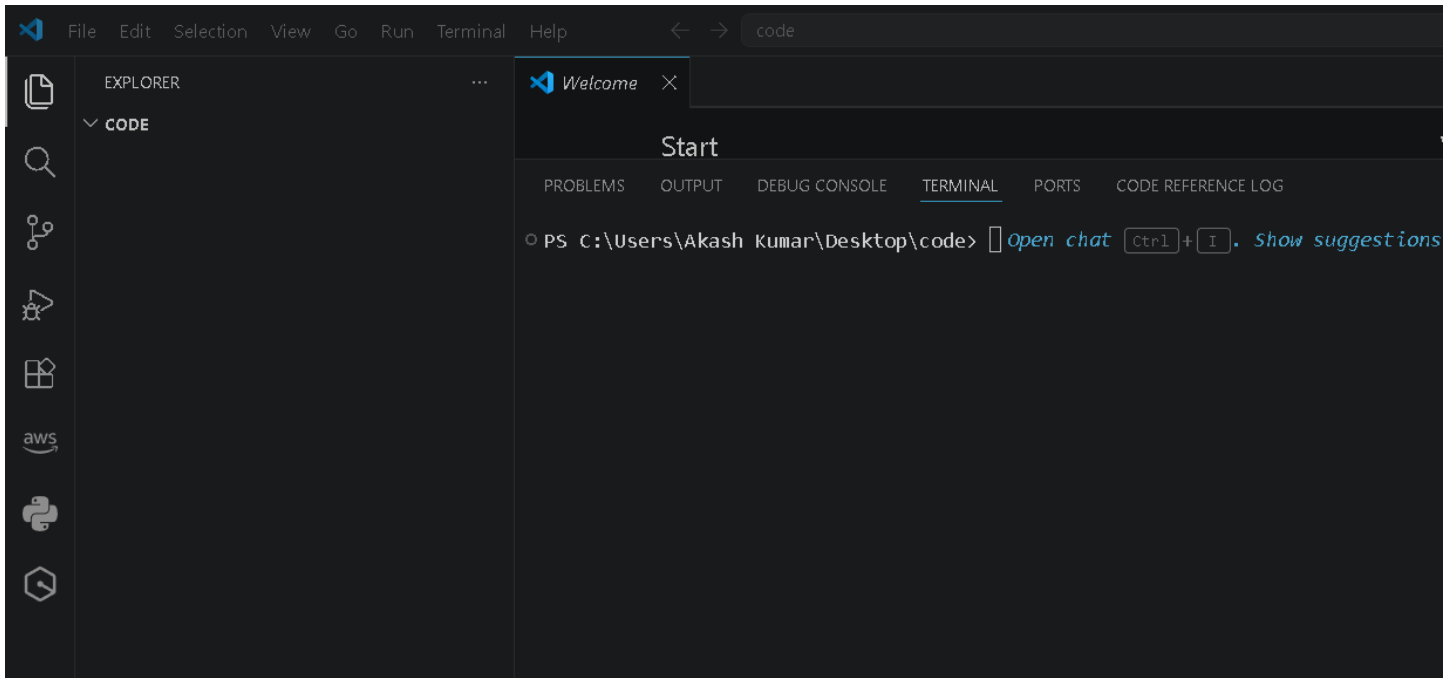


- Now append this on dev invoke URL .



- Now go inside body select raw and Text.
- And paste this

```
{  "key": "python",  "message": "implement binary search"}
```
- Now I try to send its not working because we not add boto3 layer in our code.
- Because we need latest version of boto3 because the current version as of now boto3 inside AWS Lambda does not support bedrock runtime.
- So that's why we are going to have create our own boto3 layer and the basically zip it and then upload it for lambda.
- Open your vs code.



- Now create new folder in your Vs code.
- And open terminal.

The screenshot shows the VS Code interface with the Explorer on the left showing a folder named `boto3_layer`. The terminal on the right has the `Start` tab selected. The command `mkdir boto3_layer` has been executed, and the output shows the directory listing for `C:\Users\Akash Kumar\Desktop\code`.

```
PS C:\Users\Akash Kumar\Desktop\code> mkdir boto3_layer
```

Directory: C:\Users\Akash Kumar\Desktop\code

| Mode   | LastWriteTime    | Length | Name        |
|--------|------------------|--------|-------------|
| d----- | 11-05-2026 15:53 |        | boto3_layer |

```
PS C:\Users\Akash Kumar\Desktop\code> cd boto3_layer
PS C:\Users\Akash Kumar\Desktop\code\boto3_layer>
```

- Create a new directory for the layer
  - `mkdir boto3_layer`
  - `cd boto3_layer`

The screenshot shows the VS Code interface with the Explorer on the left showing a folder named `boto3_layer`. The terminal on the right has the `Start` tab selected. The command `mkdir python` has been executed, and the output shows the directory listing for `C:\Users\Akash Kumar\Desktop\code\boto3_layer`.

```
PS C:\Users\Akash Kumar\Desktop\code\boto3_layer> mkdir python
```

Directory: C:\Users\Akash Kumar\Desktop\code\boto3\_layer

| Mode   | LastWriteTime    | Length | Name   |
|--------|------------------|--------|--------|
| d----- | 11-05-2026 15:55 |        | python |

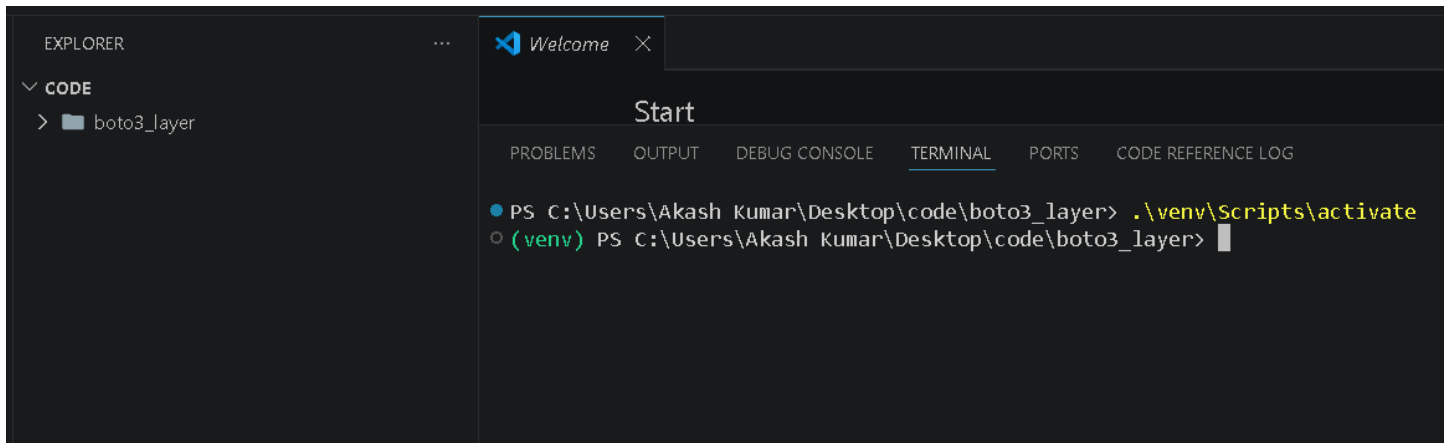
```
PS C:\Users\Akash Kumar\Desktop\code\boto3_layer>
```

- Create a python directory. AWS Lambda expects Python packages in a python directory for Python runtimes
  - `mkdir python`

The screenshot shows the VS Code interface with the Explorer on the left showing a folder named `boto3_layer`. The terminal on the right has the `Start` tab selected. The command `python -m venv venv` has been executed, and the output shows the command prompt in the `boto3_layer` directory.

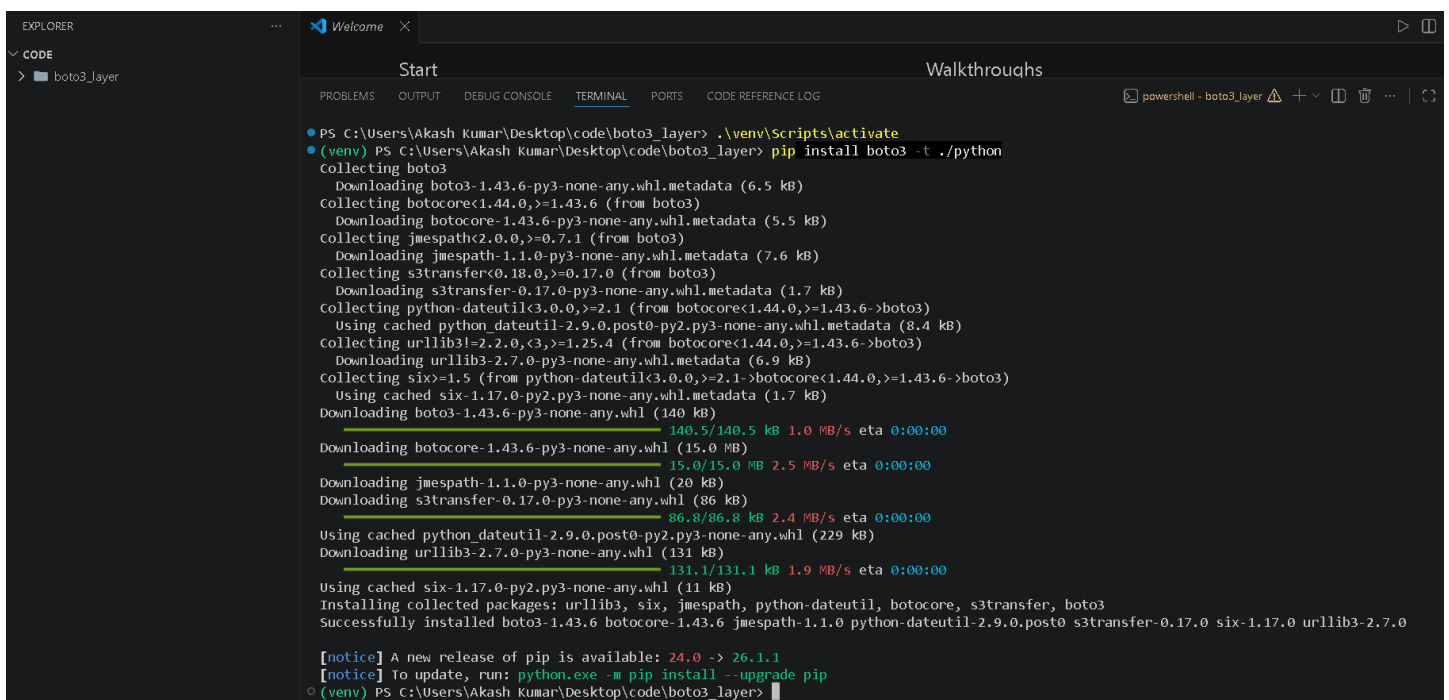
```
PS C:\Users\Akash Kumar\Desktop\code\boto3_layer> python -m venv venv
PS C:\Users\Akash Kumar\Desktop\code\boto3_layer>
```

- Create a virtual environment
  - `python -m venv venv`



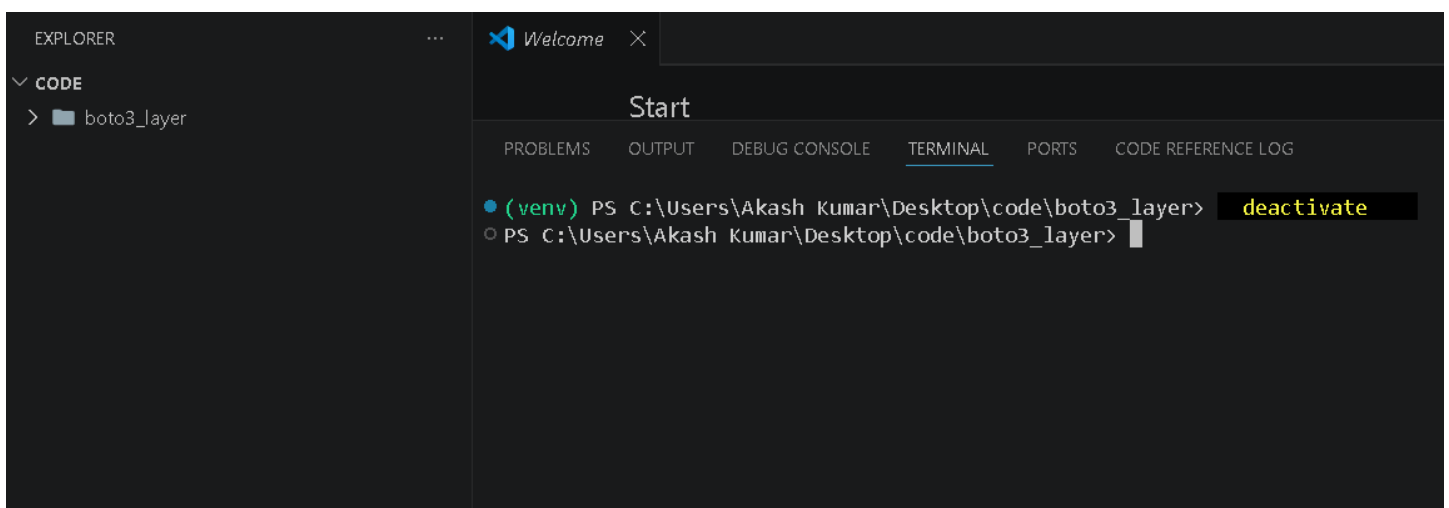
The screenshot shows the Visual Studio Code interface. On the left, the Explorer pane shows a folder named 'boto3\_layer'. The main editor area displays the 'Start' page with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, PORTS, and CODE REFERENCE LOG. The TERMINAL tab is active, showing a PowerShell prompt where the command `.\venv\Scripts\activate` has been entered and executed. The prompt now shows `(venv)` indicating the virtual environment is active.

- Activate the virtual environment
  - `.\venv\Scripts\activate`



The screenshot shows the Visual Studio Code interface with the same 'boto3\_layer' folder. The TERMINAL tab is active, showing the command `pip install boto3 -t ./python` being executed. The output shows the installation progress of boto3 and its dependencies (botocore, jmespath, s3transfer, python-dateutil, urllib3, and six). The installation is successful, and the prompt returns to `(venv)`. A notice at the bottom indicates a new release of pip is available (24.0 -> 26.1.1).

- Install the latest boto3 into the python directory
  - `pip install boto3 -t ./python`



The screenshot shows the Visual Studio Code interface. The TERMINAL tab is active, showing the command `deactivate` being entered. The prompt is still `(venv)`, but the command is highlighted in yellow, indicating it has been entered but not yet executed. The prompt below it shows `PS C:\Users\Akash Kumar\Desktop\code\boto3_layer>`.

- Deactivate the virtual environment



- deactivate

```

PS C:\Users\Akash Kumar\Desktop\code\boto3_layer> Compress-Archive -Path .\python\* -DestinationPath boto3_layer.zip
PS C:\Users\Akash Kumar\Desktop\code\boto3_layer>

```

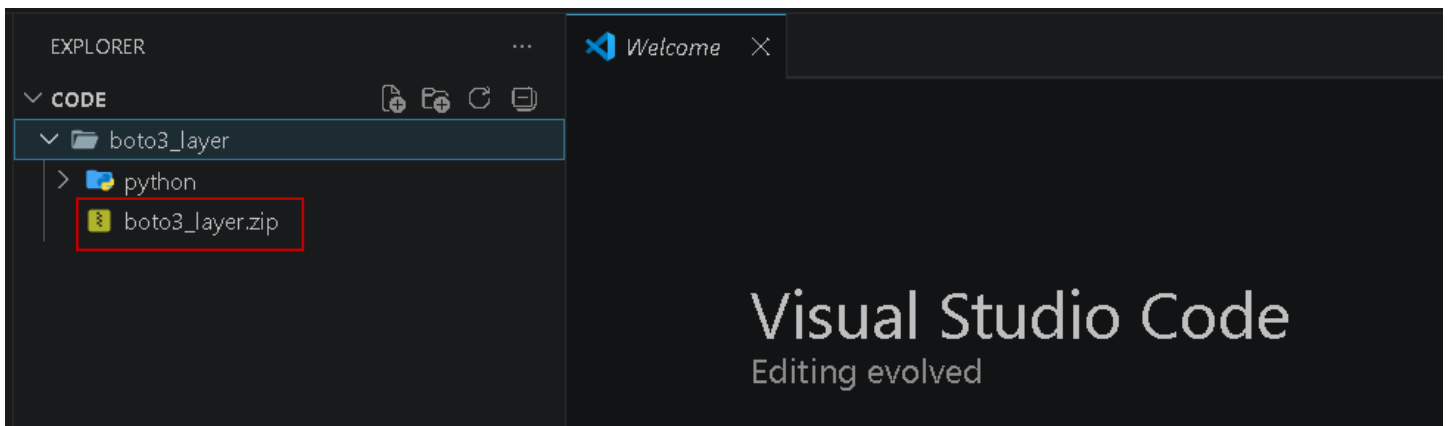
- Zip the package
  - Compress-Archive -Path .\python\\* -DestinationPath boto3\_layer.zip

```

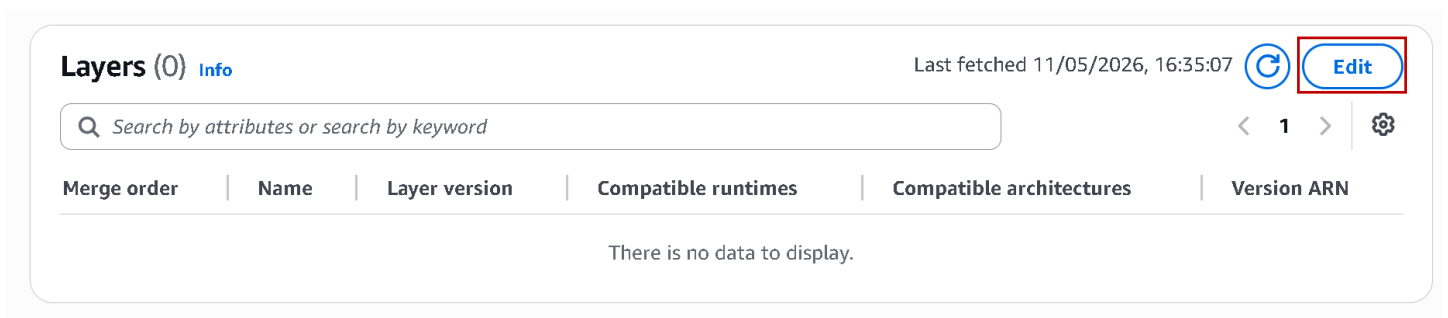
PS C:\Users\Akash Kumar\Desktop\code\boto3_layer> rm -r venv
PS C:\Users\Akash Kumar\Desktop\code\boto3_layer>

```

- Optionally, remove the virtual environment after packaging
  - rm -r venv



- Now our boto3 layer zip is ready we upload in our layer.
- Open your browser and got o your lambda function that you created.



- Now scroll down and you see a layer.

- Click on Edit.

## Edit layers

### Function runtime settings

**Runtime**  
Python 3.11

**Architecture**  
x86\_64

### Layers [Info](#)

No layers are configured on the function.

[Add a layer](#)

- Now click on add a layer.

## Choose a layer



### Layer source [Info](#)

Choose from layers with a compatible runtime and instruction set architecture or specify the Amazon Resource Name (ARN) of a layer version. You can also [create a new layer](#).

☐ **AWS layers**  
Choose a layer from a list of layers provided by AWS.

☒ **Custom layers**  
Choose a layer from a list of layers created by your AWS account.

☐ **Specify an ARN**  
Specify a layer by providing the ARN.

### Custom layers

Layers created by your AWS account.



[Cancel](#)

[Add](#)

- Click on create a new layer.

## Layer configuration

### Name

### Description - *optional*

- ☒ Upload a .zip file
- ☐ Upload a file from Amazon S3

[Choose file](#)

For files larger than 10 MB, consider uploading using Amazon S3.

- Enter a name of layer.
- And then choose a file.

**Compatible runtimes - optional** | [Info](#)  
Choose up to 15 runtimes.

Search Compatible runtimes ▼ 

Python 3.11 ✕ Python 3.10 ✕

**License - optional** | [Info](#)

[Cancel](#) [Create](#)

- Now choose compatible runtimes.
- And then simply click Create.

☰ [Lambda](#) > [Layers](#) > [boto3\\_bedrock](#) > 1 ⓘ 🖨

✔ Successfully created layer boto3\_bedrock version 1. ✕

### Version details

|                                      |                                                                                                                                                                     |                                                      |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| <b>Version</b><br>1                  | <b>Version ARN</b><br> arn:aws:lambda:us-west-2:463646775279:layer:boto3_bedrock:1 | <b>Description</b><br>-                              |
| <b>Created</b><br>1 second ago       | <b>License</b><br>-                                                                                                                                                 | <b>Compatible runtimes</b><br>python3.11, python3.10 |
| <b>Compatible architectures</b><br>- |                                                                                                                                                                     |                                                      |

- Our layer is ready.
- Now go back to the lambda function that you created.
- In lambda function scroll down in layer click Edit.
- Then click Add a Layer.

### Choose a layer

**Layer source** | [Info](#)  
Choose from layers with a compatible runtime and instruction set architecture or specify the Amazon Resource Name (ARN) of a layer version. You can also [create a new layer](#).

☐ AWS layers  
Choose a layer from a list of layers provided by AWS.

☒ Custom layers  
Choose a layer from a list of layers created by your AWS account.

☐ Specify an ARN  
Specify a layer by providing the ARN.

**Custom layers**  
Layers created by your AWS account.

boto3\_bedrock ▼ 

**Version**  
1 ▼ 

**Description**  
-

[Cancel](#) [Add](#)

- Now choose Custom Layer.
- And select your created Layer and version.
- And simply click ADD.

**Layers** [Info](#)

boto3\_bedrock

arn:aws:lambda:us-west-2:463646775279:layer:boto3\_bedrock:1

Add a layer

Some Lambda layers may not be compatible with functions running on Lambda Managed Instances. Verify layer compatibility and remove any incompatible layers from your function configuration. Layer compatibility depends on the runtime and architecture of your EC2 instance.

Cancel
Save

- Now simply click Save.

Code
Test
Monitor
**Configuration**
Aliases
Versions

**Configuration**

General configuration
Triggers
Permissions
Destinations
Function URL

**General configuration** [Info](#)

|                               |                                               |                                    |
|-------------------------------|-----------------------------------------------|------------------------------------|
| <b>Description</b><br>-       | <b>Memory</b><br>128 MB                       | <b>Ephemeral storage</b><br>512 MB |
| <b>Timeout</b><br>0 min 3 sec | <b>SnapStart</b> <a href="#">Info</a><br>None |                                    |

Edit

- Scroll up.
- And go inside configuration and click General configuration.
- And then click edit.

**Timeout**

4 min 3 sec

**Execution role**

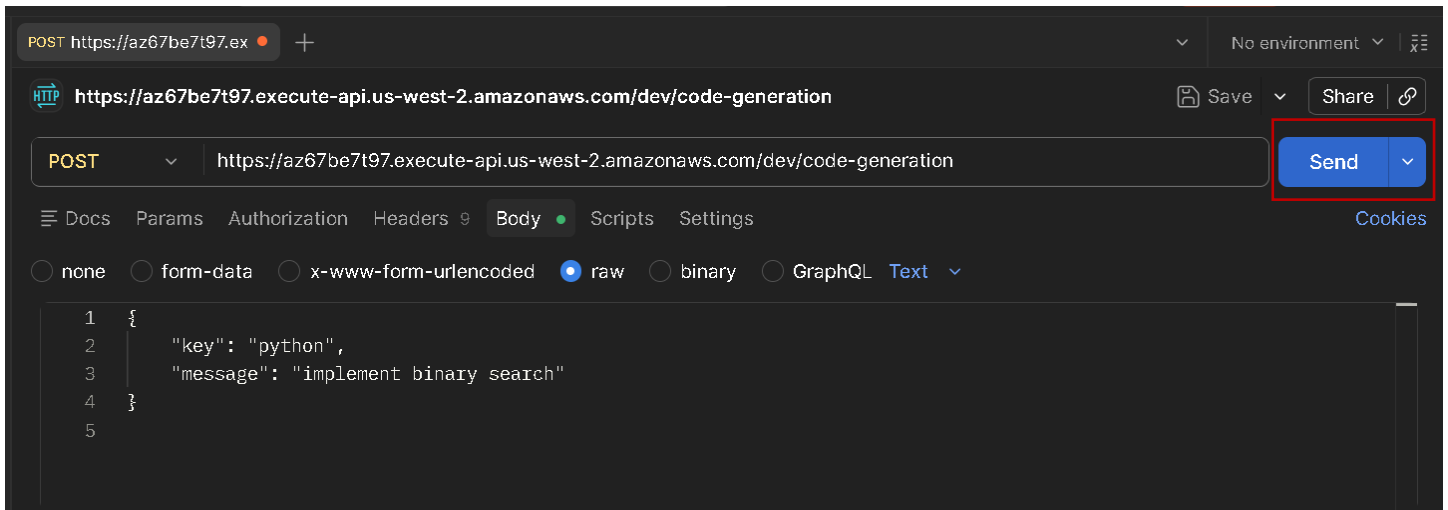
To access other AWS services and resources your function needs an IAM role. Choose a role with the right permissions for your function.

bedrock\_code\_generation-role-jy2t52b5
Create new role

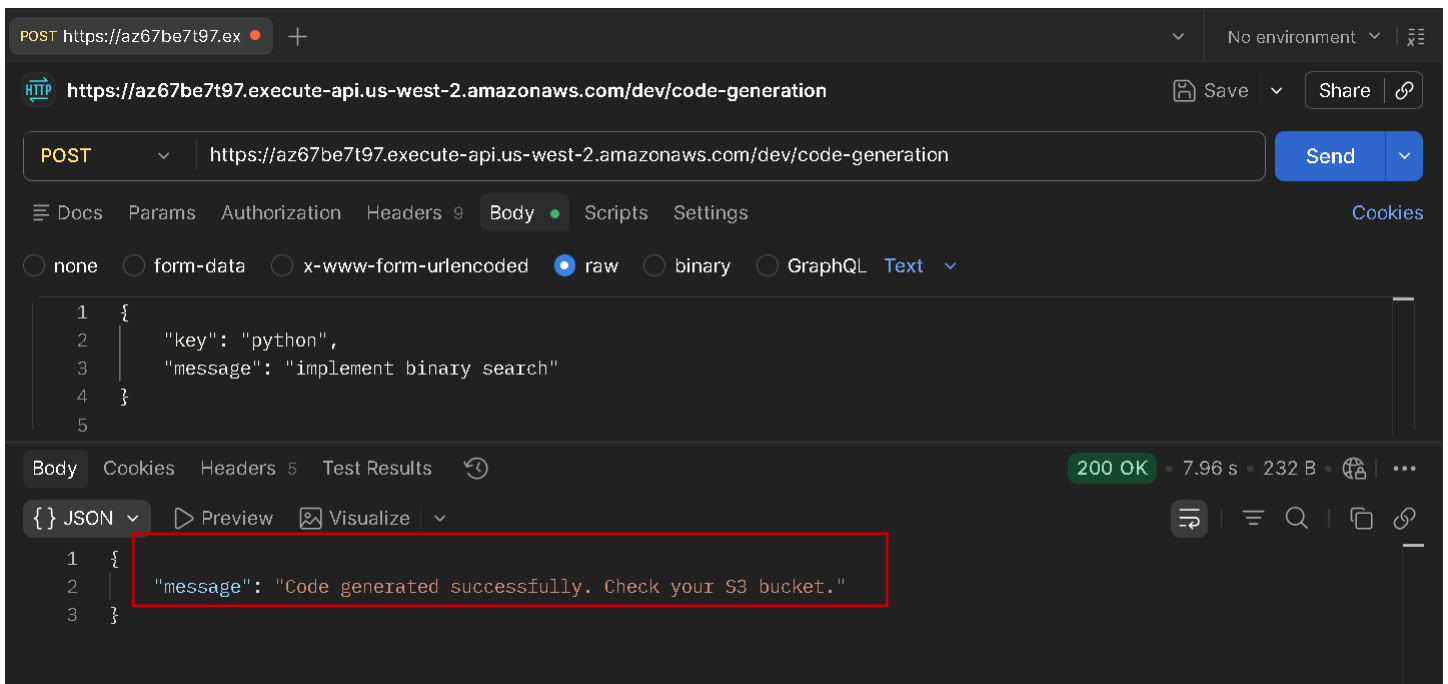
[View role details in IAM](#)

Cancel
Save

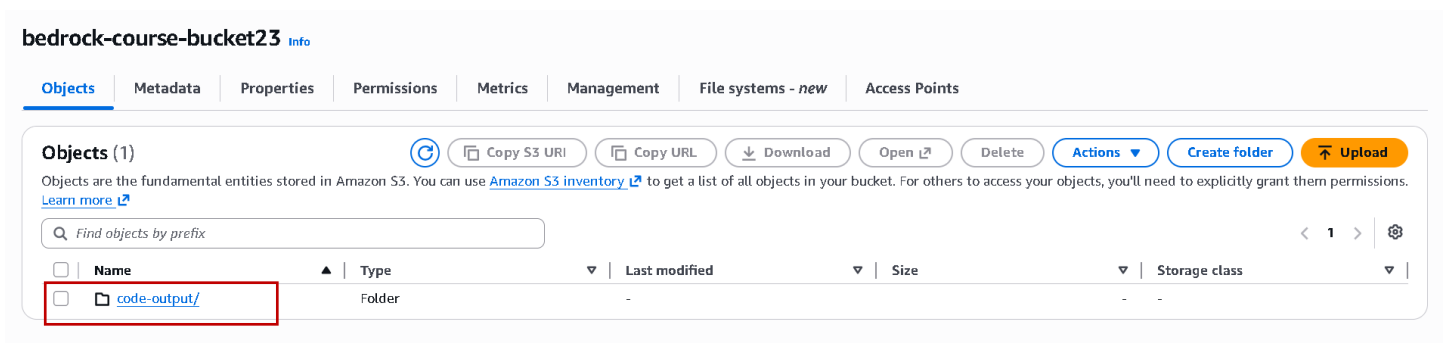
- Now simply scroll down and make time out as 4min 3sec.
- And leave everything and simply click save.



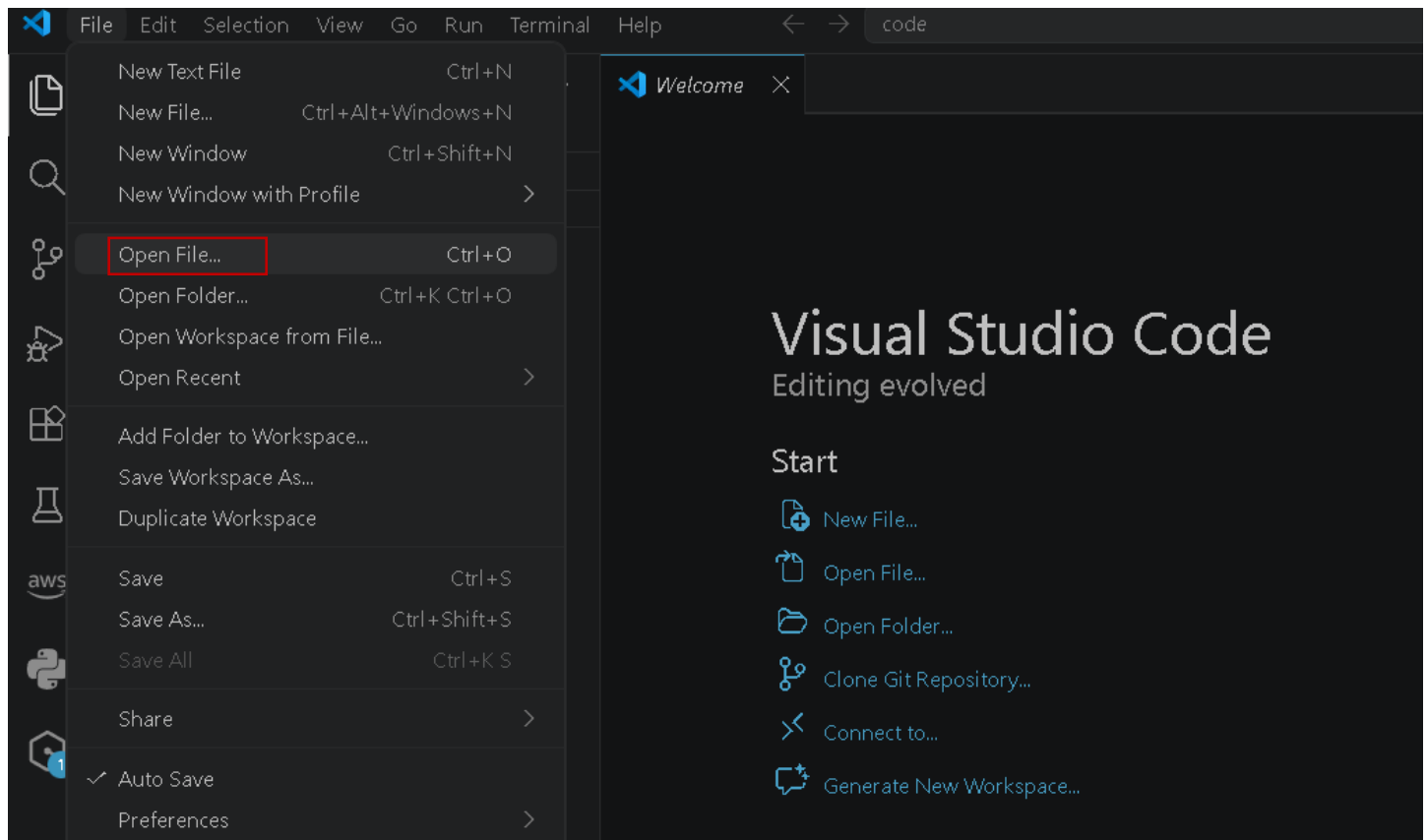
- Now go back to your postman tab and simply click Send.



- Now Result is Code generation.
- Now go to your S3 bucket and check our code is generated or not.



- Now our code is generated and saved in s3 bucket.
- Now download and check.



- Now open Vs Code and click Open File.
- And select downloaded file.

```
120857.python X
C: > Users > Akash Kumar > Downloads > 120857.python
1  def binary_search(arr, target):
2      left, right = 0, len(arr) - 1
3
4      while left <= right:
5          mid = (left + right) // 2
6
7          if arr[mid] == target:
8              return mid
9          elif arr[mid] < target:
10             left = mid + 1
11          else:
12             right = mid - 1
13
14     return -1
15
16
17 if __name__ == "__main__":
18     sorted_array = [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
19
20     print(binary_search(sorted_array, 7))
21     print(binary_search(sorted_array, 1))
22     print(binary_search(sorted_array, 19))
23     print(binary_search(sorted_array, 6))
```

- Our code is generated perfectly.